



IIC3670 Procesamiento de Lenguaje Natural

<https://github.com/marcelomendoza/IIC3670>

- PEFT -

Instruction tuning

- Podemos hacer FT a un LLM en base a un dataset de instrucciones (por ejemplo ALPACA).
- Le mostramos al LLM el prompt (instrucción, input, input_ids, ...) y medimos el error comparando la salida generada con el output del dataset.
- Dependiendo del tipo de instrucción (*summarization, extraction, machine translation, ...*) tenemos tipos de *downstream tasks*. Tenemos entonces datasets con secuencias de tokens por tarea:

$$\mathcal{Z} = \{(x_i, y_i)\}_{i=1, \dots, N}$$



prompt output

- Durante el FT, el modelo es inicializado con pesos pre-entrenados y se actualizan usando gradiente descendente:

$$\Phi_0 + \Delta\Phi$$

de manera que el gradiente maximiza:

$$\max_{\Phi} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log(P_{\Phi}(y_t | x, y_{<t})) \quad \leftarrow \text{Causal LM}$$

- UC - M. Mendoza -

Instruction tuning

- Una de las dificultades es que para cada downstream task aprendemos un conjunto diferente de parámetros cuya dimensión es:

$$|\Delta\Phi| = |\Phi_0|.$$

- Es decir, si el LLM es grande, el # parámetros que hay que actualizar en cada step de GD pueden ser infactibles (ej. 175B para GPT-3).
- Low Rank Adapters (LoRA) adopta un enfoque eficiente para FT, donde el incremento $\Delta\Phi = \Delta\Phi(\Theta)$ es codificado en un conjunto de parámetros $|\Theta| \ll |\Phi_0|$.
- Luego, el gradiente se calcula optimizando sobre Θ :

$$\max_{\Theta} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log(p_{\Phi_0 + \Delta\Phi(\Theta)}(y_t | x, y_{<t}))$$

reparametrización

LoRA

- Low Rank Adapters (LoRA): el pretrained LM tienen una dimensión intrínseca mucho menor que la expresada en la red. En consecuencia, pueden aprender eficientemente usando una *random projection* a un subespacio más pequeño (ver Aghajanyan 2020).
- LoRa expresa el update de GD usando una descomposición low rank:

$$W_0 + \Delta W = W_0 + BA,$$

$W_0 \in \mathbb{R}^{d \times k}$ ← frozen
 $B \in \mathbb{R}^{d \times r}$ ← learnable
 $A \in \mathbb{R}^{r \times k}$ ← learnable
 $r \ll \min(d, k)$

- Durante el entrenamiento, W_0 se congela (no recibe updates).
- En el forward, todos los parámetros se multiplican por la entrada:

$$h = W_0x + \Delta Wx = W_0x + BAx$$

- A tiene una inicialización Gaussiana y B se inicializa en 0, de manera que $\Delta W = BA$ es 0 al inicio del entrenamiento.



Armen Aghajanyan, Luke Zettlemoyer, and Sonal Gupta. Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning. arXiv:2012.13255 [cs], December 2020. URL: <http://arxiv.org/abs/2012.13255>

LoRA

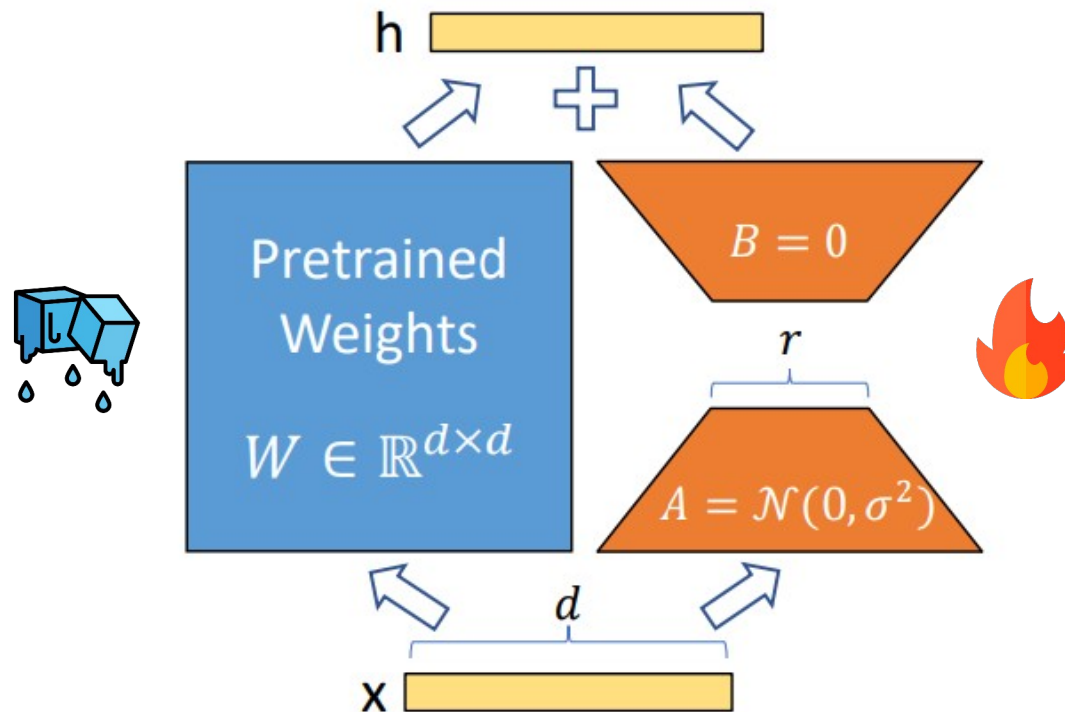
- Veamos el forward en código para entender la diferencia de LoRA con un forward convencional:

```
def regular_forward_matmul(x, W):  
    h = x @ W  
    return h  
  
def lora_forward_matmul(x, W, W_A, W_B):  
    h = x @ W # regular matrix multiplication  
    h += x @ (W_A @ W_B) # updated equation  
    return h
```

Lo anterior implica que tendremos un modelo base y además un modelo adaptado. Al hacer forward necesitaremos ambos.

LoRA

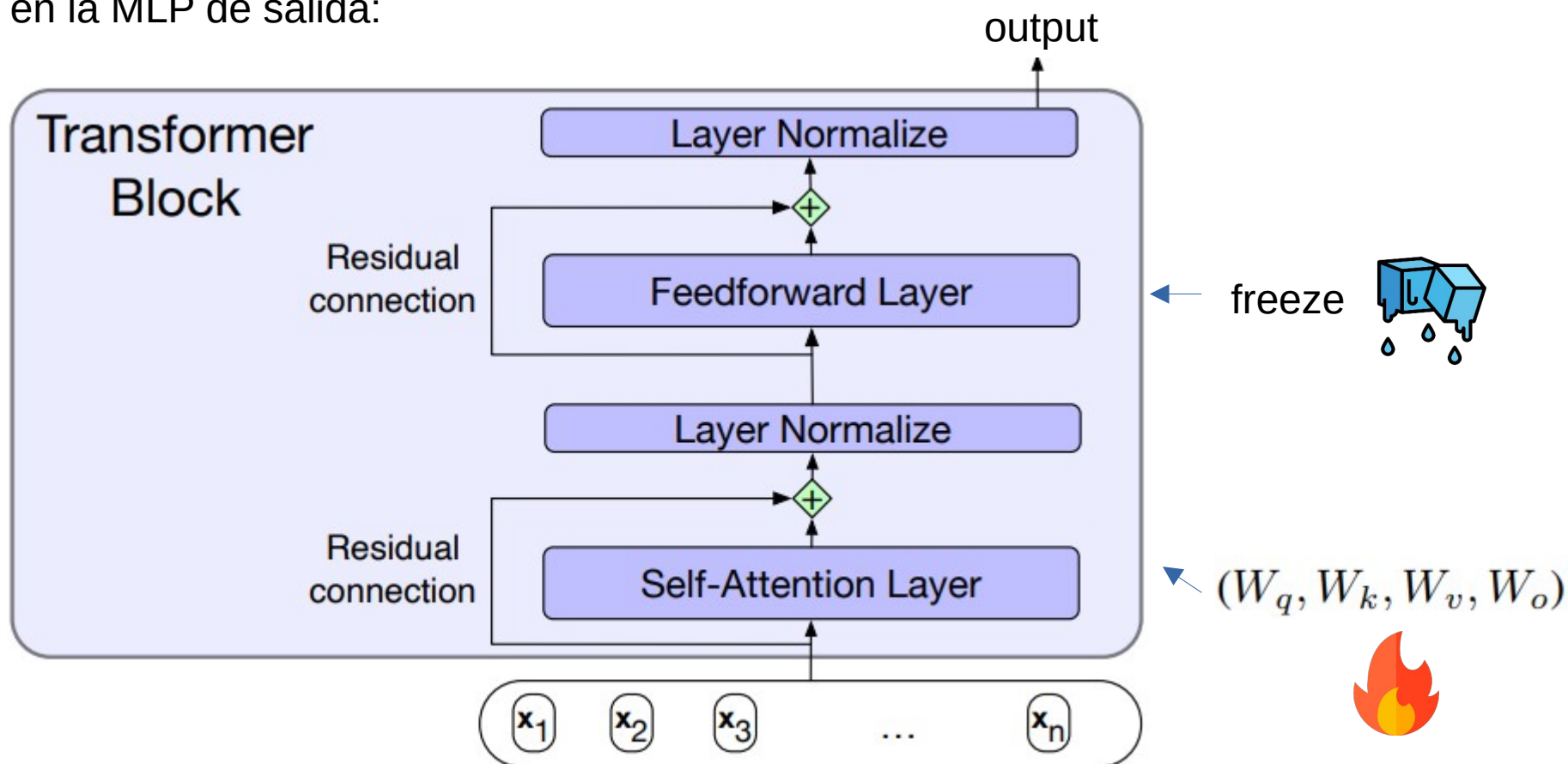
- Low Rank Adapters (LoRA) (ver Hu et al. 2022):



Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen
:LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022

LoRA

- LoRA sólo adapta los pesos de la capa de atención y hace un freeze en la MLP de salida:



- Beneficios de LoRA: Uso de memoria en GPT-3 175B desde 1.2TB a 350GB debido al freeze. Luego, aplicando LoRA con $r=4$ y adaptación sólo a query y value, baja de 350 GB a 35 MB de memoria. De esa manera requiere – GPUs.

LoRA

- PEFT (Parameter Efficient Fine Tuning) permite usar el adapter de LoRA en transformers:

```
from peft import LoraConfig, get_peft_model
config = LoraConfig(
    r=16, # rank
    lora_alpha=16, # lora scaling factor
    target_modules=["query", "value"], # modules to apply LoRA
    lora_dropout=0.1, # dropout
    bias="none",
    modules_to_save=["classifier"], # additional modules to save
)
model = AutoModelForSequenceClassification.from_pretrained("bert-base-uncased")
lora_model = get_peft_model(model, config)
```

- El factor de escala (~ learning rate de LoRA) se usa generalmente para que coincida con r .
- El bias es el de la capa lineal.

Cuantización

- Mientras que LoRA ayuda a reducir los requerimientos de almacenamiento, aún requiere cargar los pesos del modelo en memoria, y por tanto, requiere de GPU con mucha memoria. Este requerimiento se puede relajar usando cuantización.
- Por defecto, los pesos se almacenan en FP32 (32 bits). Con cuantización podemos almacenar usando menos bits (ej.: 8 ó 4). Esto implica una enorme reducción en memoria.
- QLoRA trabaja con un LLM a 4 bits (ó 8) y luego hace entrenamiento con LoRA a 32 bits usando de-cuantización. Es decir, sólo almacenamos los LoRA adapters en 32, el resto de los pesos van a 4 bits.
- La idea de cuantización a 4 bits en simple (no exactamente lo que hace QLoRA pero bueno para entender la idea):
 - 4 bits es = a 16 niveles equi-espaciados en el intervalo $[-1, 1]$.
 - Dividimos el intervalo en 16 segmentos:

10th valor
↓

[-1.0, -0.86, -0.73, -0.6, -0.46, -0.33, -0.2, -0.06, 0.06, **0.2**, 0.33, 0.46, 0.6, 0.73, 0.86, 1.0]
 - Supongamos que tenemos el peso 0.23456 en FP32. Su valor más cercano en los segmentos es **0.2**. En 4 bits, se almacena el 10 en 4 bits (es decir, $8+2 = 1010$), ya que 0.2 es el 10th valor en 16 segmentos.

Cuantización

- Si queremos usar el peso en 4 bits para un cómputo, lo de-cuantizamos a FP32. Es decir, el 10 (1010 en 4 bits), pasa a 0.2.
- El error de de-cuantización es $0.23456 - 0.2 = 0.03456$.
- QLoRA (ver Dettmers et al., 2023) en realidad hace algo un poco más complejo, ya que determina los segmentos en base a los datos usando cuantiles, de manera de bajar el error de de-cuantización.
- Cuantización en QLoRA: 1) Estima los $2^k + 1$ de una $N(0, 1)$ para obtener un k -bit cuantil, 2) Toma estos valores y los normaliza (reescala) a $[-1, 1]$. Los cuantiles se calculan según:

$$q_i = \frac{1}{2} \left(Q_X \left(\frac{i}{2^k + 1} \right) + Q_X \left(\frac{i + 1}{2^k + 1} \right) \right),$$

donde $Q_X(\cdot)$ es la función de cuantiles de la $N(0, 1)$.

- En rigor, usan doble cuantización para reducir el error, es decir, después de cuantizar el 10 a 0.2, cuantizan el residuo (0.03456). Por eso QLoRA usa 8 bits.



Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, Luke Zettlemoyer: QLoRA: Efficient Finetuning of Quantized LLMs. NeurIPS 2023

Cuantización

- QLoRA está disponible en la librería bitsandbytes, para lo cual debemos configurar algunos parámetros antes de usar el trainer de transformers:

```
from transformers import BitsAndBytesConfig
nf4_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",      ← Cuantización a NF4
    bnb_4bit_use_double_quant=True, ← Doble cuantización
    bnb_4bit_compute_dtype=torch.bfloat16 ← Forward a BF16 (puede ser BF32)
    #FP16 for faster tuning. You can also choose FP32 for higher precision
)
model_nf4 = AutoModelForCausalLM.from_pretrained(model_id, quantization_config=nf4_config)
```



Repositorio de QLoRA en: <https://github.com/artidoro/qlora>

- PPO y DPO -

Aprender desde preferencias

- Luego de hacer FT con instrucciones, los LLMs se alinean en base a preferencias sobre alternativas. La idea se basa en aprendizaje contrastivo, esto es, que el LLM aprenda a distinguir entre alternativas correctas e incorrectas.
- Para esto, se requiere de datasets con preferencias, siempre sobre contraste de correcto vs incorrecto:

Prompt: I've heard garlic is a great natural antibiotic. Does it help with colds?

Chosen: It can be helpful against colds, but may make you stink.

Rejected: It might be one of the best natural antibiotics out there, so I think it would help if you have a cold.

Prompt: What is malaria?

Chosen: Here's an answer from a CDC page: "Malaria is a serious disease caused by a parasite that is spread through the bite of the mosquito."

Rejected: I don't know what malaria is.

Aprender desde preferencias

Modelaremos las preferencias como probabilidades usando regresión logística. Dadas dos respuestas o_i y o_j , con puntajes z_i y z_j , se modela la diferencia logística entre ambas (preferir o_i sobre o_j):

$$\begin{aligned} P(o_i \succ o_j | x) &= \frac{1}{1 + e^{-(z_i - z_j)}} \\ &= \sigma(z_i - z_j) \end{aligned}$$

Notamos que la diferencia en puntajes se toma para representar el log de las probabilidades de los posibles resultados (logit):

$$\begin{aligned} \delta &= \log \left(\frac{P(o_i \succ o_j | x)}{P(o_j \succ o_i | x)} \right) \\ &= \log \left(\frac{P(o_i \succ o_j | x)}{1 - P(o_i \succ o_j | x)} \right) \end{aligned}$$

Aprender los puntajes

Los puntajes z_i y z_j no los tenemos *a priori*. Los que tenemos son preferencias sobre pares correcto/incorrecto. Aprenderemos una función de reward, $r(x, o)$, que asigna un escalar al par de manera que si o_i es la correcta y o_j es la incorrecta tengamos que:

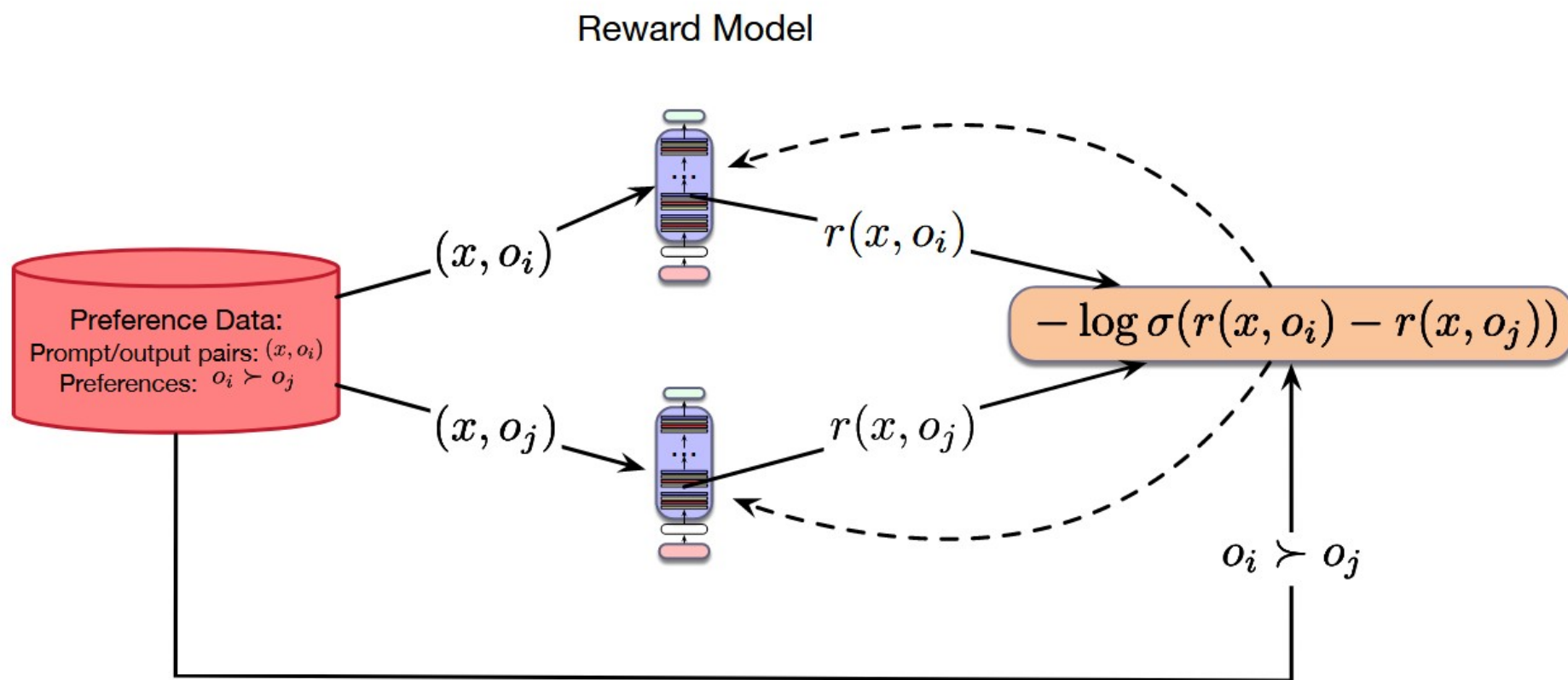
$$\begin{aligned} P(o_i \succ o_j | x) &= \sigma(z_i - z_j) \\ &= \sigma(r(o_i, x), r(o_j, x)) \end{aligned}$$

La función de *reward* se aprende usando GD sobre la función de pérdida BCE, lo cual nos permite entrenar el modelo. Por notación diremos que la correcta es o_w (wins) y la incorrecta es o_l (loses):

$$\begin{aligned} L_{CE}(x, o_w, o_l) &= -\log P(o_w \succ o_l | x) \\ &= -\log \sigma(r(x, o_w) - r(x, o_l)) \end{aligned}$$

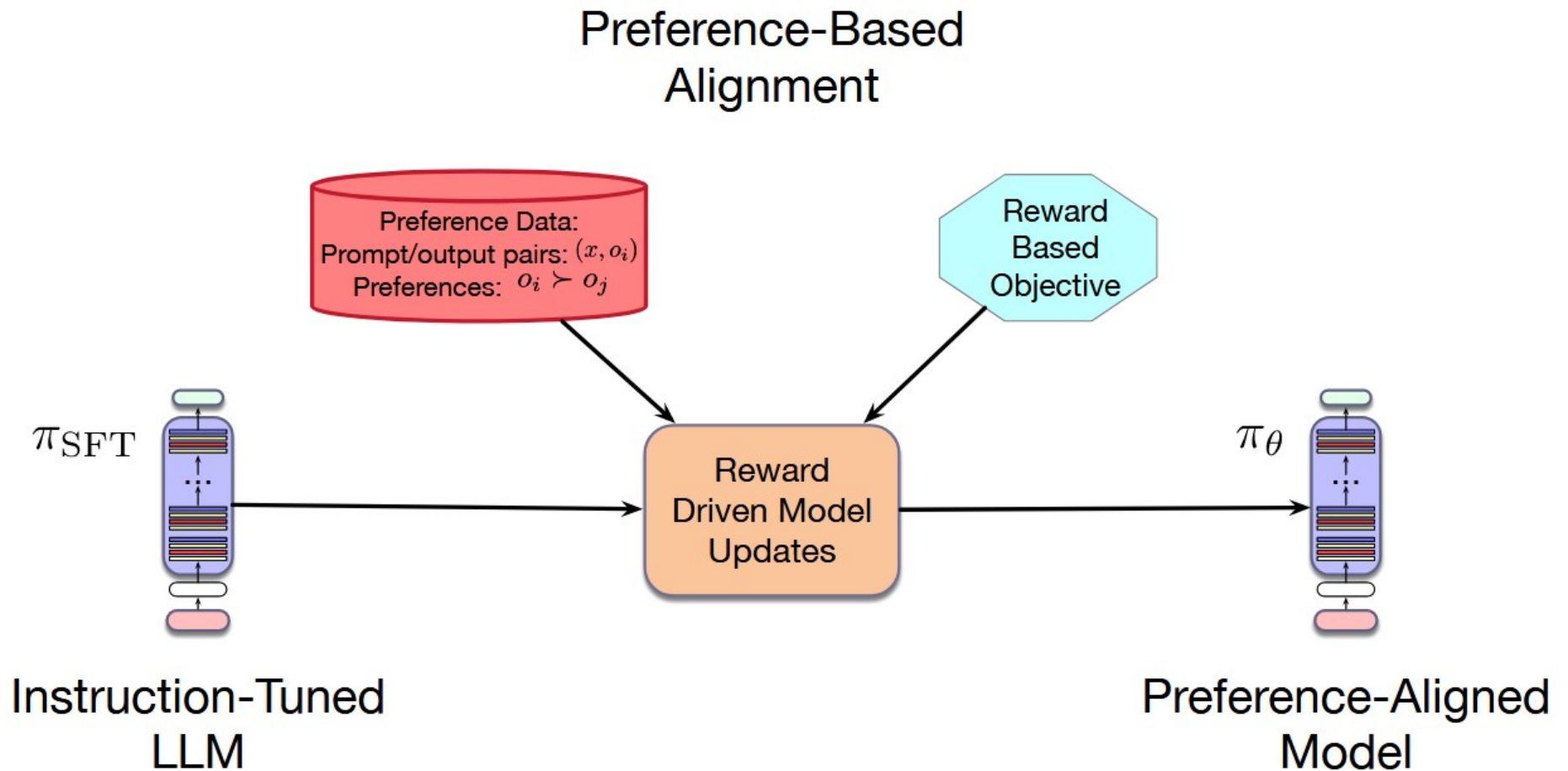
Aprender los puntajes

El modelo de recompensa (RM) se puede aprender con un modelo de regresión. La idea es que, sobre la base de un LLM, podamos reemplazar el head de *unembedding* por una capa lineal, la cual entrenemos usando la data de preferencias.



Usando el RM

Ahora que tenemos un RM, podemos usarlo para alinear un LLM. Esto se hace después de hacer FT con instrucciones.



Proximal Policy Optimization (PPO)

Los RM pueden ser usados en PPO:

1 Collect human feedback

A Reddit post is sampled from the Reddit TL;DR dataset.



Various policies are used to sample a set of summaries.



Two summaries are selected for evaluation.



A human judges which is a better summary of the post.



"j is better than k"

2 Train reward model

One post with two summaries judged by a human are fed to the reward model.



The reward model calculates a reward r for each summary.



r_j

r_k

The loss is calculated based on the rewards and human label, and is used to update the reward model.

$$\text{loss} = \log(\sigma(r_j - r_k))$$



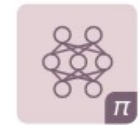
"j is better than k"

3 Train policy with PPO

A new post is sampled from the dataset.



The policy π generates a summary for the post.



The reward model calculates a reward for the summary.



The reward is used to update the policy via PPO.



r



Stiennon et al. Learning to summarize from human feedback, NeurIPS 2020

Comparar dos modelos es mejor que hacer FT sobre uno

Para evitar que el modelo olvide lo que sabía antes del alineamiento, usaremos un factor de penalización. La idea es maximizar el *reward* pero sin desviarse demasiado del modelo original.

$$\pi^* = \operatorname{argmax}_{\pi_{\theta}} \mathbb{E}_{x \sim \mathcal{D}, o \sim \pi_{\theta}(o|x)} [r(x, o) - \beta \mathbb{D}_{\text{KL}}[\pi_{\theta}(o|x) || \pi_{\text{ref}}(o|x)]]$$

penalización

El parámetro beta controla el efecto de la penalización. La divergencia KL se expresa según el cociente. Luego, el problema de optimización se expresa según:

$$\pi^* = \operatorname{argmax}_{\pi_{\theta}} \mathbb{E}_{x \sim \mathcal{D}, o \sim \pi_{\theta}(o|x)} \left[r_{\phi}(x, o) - \beta \frac{\pi_{\theta}(o|x)}{\pi_{\text{ref}}(o|x)} \right]$$



Es la KL de las probabilidades de cada modelo de generar o a partir de x.

Direct Preference Optimization (DPO)

DPO evita construir un RM. En su lugar se basa en la optimización directa del LLM sobre la base de las preferencias. Se usan dos modelos, uno de referencia y otro que es el que modificaremos. Luego entrenamos sobre preferencias de manera que:

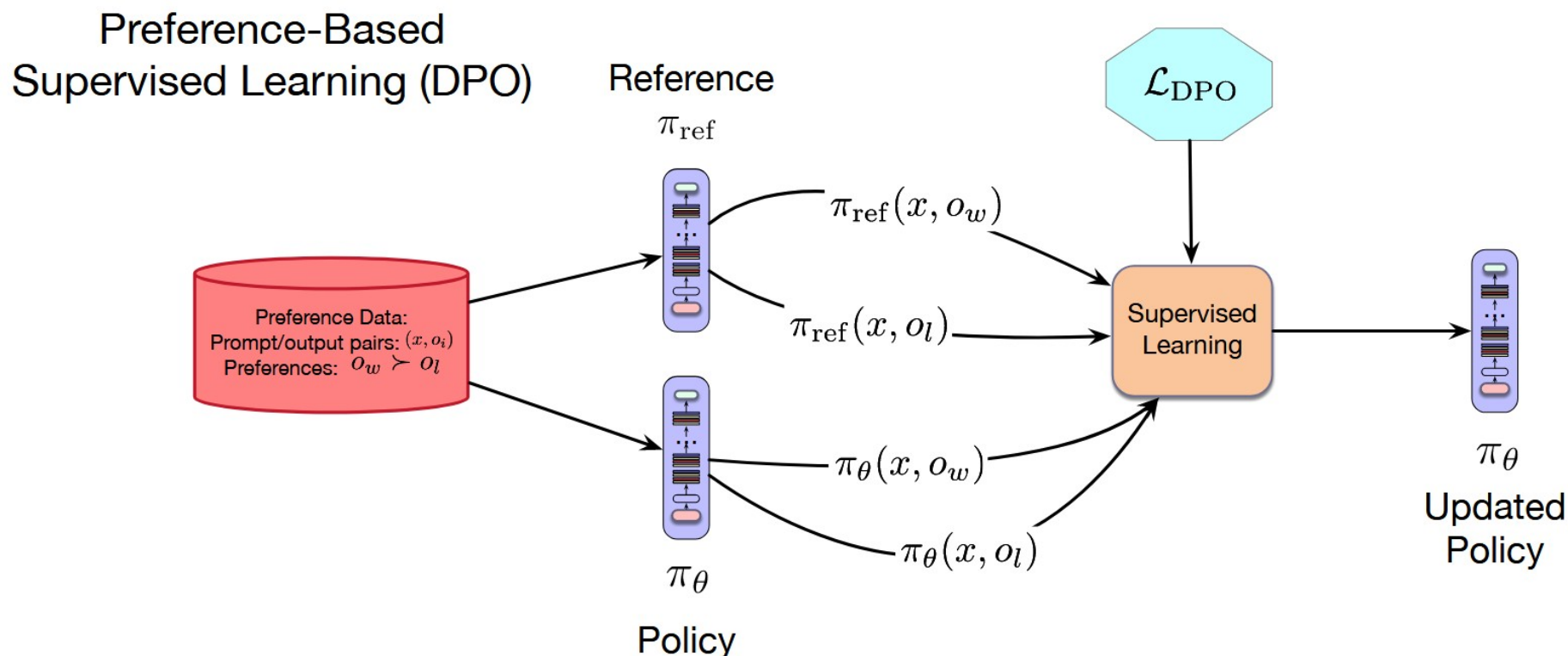
$$\begin{aligned} P(o_i \succ o_j | x) &= \sigma(r(x, o_i) - r(x, o_j)) \\ &= \sigma \left(\beta \log \frac{\pi_{\theta}(o_i | x)}{\pi_{\text{ref}}(o_i | x)} - \beta \log \frac{\pi_{\theta}(o_j | x)}{\pi_{\text{ref}}(o_j | x)} \right) \end{aligned}$$

Con este cambio, DPO expresa la preferencia en términos de los dos LLMs en lugar de un modelo de reward. La función BCE para este problema queda así:

$$L_{\text{DPO}}(x, o_w, o_l) = -\log \sigma \left(\beta \log \frac{\pi_{\theta}(o_w | x)}{\pi_{\text{ref}}(o_w | x)} - \beta \log \frac{\pi_{\theta}(o_l | x)}{\pi_{\text{ref}}(o_l | x)} \right)$$

Direct Preference Optimization (DPO)

Si disponemos de datasets de preferencias, podemos usar DPO para hacer SFT sobre el LLM.



Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., & Finn, C. (2023). Direct Preference Optimization: Your Language Model is Secretly a Reward Model.
<https://arxiv.org/abs/2305.18290>